SOLO OPERATING

Cell จังหวะการทำงาน: รอบ (cycle), งานที่ผูกรอบ, KPI และ review ของ Solo Empire

Product Card

รหัสเอกสาร	CARD-solo-operating
ฉบับ	0.1
วันที่	2026-10-11
สถานะ	ร่างเพื่อพิจารณา

เอกสารภายในฉบับร่าง — ใช้ประกอบการพัฒนาและดูแลระบบ

สารบัญ

1. สรุปคุณค่าในหนึ่งประโยค
2. ปัญหาและผู้ใช้เป้าหมาย
3. สิ่งที่ทำได้จริงตอนนี้
4. หลักฐานสถานะ
5. คุณค่าและจุดต่าง
6. ช่องว่างสำคัญ
7. คำตัดสิน
8. ขั้นต่อไป
การอนุมัติ
แหล่งข้อมูล

solo-operating — Product Card

1. สรุปคุณค่าในหนึ่งประโยค

ช่วยเจ้าของ Solo Empire วางรอบการทำงาน (sprint, weekly, planning, review) ผูกงานกับเป้าหมาย และวัด KPI ต่อรอบ ผ่าน API ของ cell เอง

2. ปัญหาและผู้ใช้เป้าหมาย

- ปัญหา: ข้อมูลรอบงานและ KPI เดิมอยู่ใน control plane ส่วนกลาง
- ผู้ใช้: เจ้าของคนเดียว (internal)

3. สิ่งที่ทำได้จริงตอนนี้

- API `/v1/cycles`, `/v1/cycle-tasks`, `/v1/cycle-goal-links`, `/v1/cycle-reviews`, `/v1/kpis`, `/v1/kpi-measurements` (list, stats, get, create, patch, delete) สร้างจาก `cell.yaml` ด้วย solo-cell-kit
- Publisher `operating-publisher` (cron `*/15 * * * *`) ตาม README
- `task_id` และ `goal_id` อ้าง cell อื่นเป็น id ธรรมดา (ไม่มี foreign key ข้าม cell)

4. หลักฐานสถานะ

รายการ	ค่า (ตรวจจาก repository)
Stack	FastAPI, Python
Commits	2 (แรก 2026-10-10 → ล่าสุด 2026-10-11)
Source	ประมาณ 2,213 บรรทัดใน 15 ไฟล์ (ไม่รวมเทสต์) — ส่วนใหญ่มาจาก template cellkit ร่วมกัน
Tests	2 ไฟล์
CI	ไม่มี
Deploy config	Dockerfile + Railway (ตาม README); ไม่ได้ตรวจ deploy จริงในรอบนี้
Live URL	ไม่มี (ไม่ได้ตรวจ)
README	40 บรรทัด

5. คุณค่าและจุดต่าง

- แยกเป็น 1 cell = 1 repo / 1 backend / 1 Neon database ตามแนว decentralize ของ Solo Empire; ใช้ identity, authorization, executions และ audit ของ `empire-platform` ผ่าน HTTP ด้วย token ของผู้เรียก ไม่เก็บ provider token เอง
- มุมรายได้: ไม่มี — เป็นโครงสร้างภายในของเจ้าของ แต่ pattern สร้าง cell จาก `cell.yaml` ด้วย `solo-cell-kit` ใช้ซ้ำได้ **[สมมติฐาน]**

6. ช่องว่างสำคัญ

- ไม่มี CI
- โค้ดเกือบทั้งหมดมาจาก template `cellkit` ร่วมกับอีก 10 repo โครงเดียวกัน (structure signature ตรงกัน) — ส่วนที่เป็นของ domain นี้คือ `cell.yaml`
- มีข้อมูลจริงใน Neon หรือไม่ **[ต้องยืนยัน]**

7. คำตัดสิน

`showcase` — ระบบภายในที่ใช้งานได้ตามสัญญา API แต่เป็น instance ของ template ไม่ใช่ผลิตภัณฑ์ จะเปลี่ยนเมื่อมีผู้ใช้นอกเหนือจากเจ้าของ

8. ขั้นต่อไป

1. ยืนยันว่า API ออนไลน์และมีข้อมูลจริง
2. เพิ่ม CI รัน `pytest`

การอนุมัติ

บทบาท	สถานะ	วันที่	ลงนาม
Product owner (Book Dev)	รอพิจารณา	—	_

แหล่งข้อมูล

- `README.md`
- `cell.yaml`
- `tests/test_contract.py`, `migrations/`, `Dockerfile`